



JavaScript Arrays and Collections

Course Overview

Course Overview

Hey everyone. My name is Jeff Batt, and welcome to this course on JavaScript Arrays and Collections. I am the founder and head trainer at Learning Dojo. Do you understand the basics of arrays, but you want to know where to go next, or are you looking for ways to better store and to work with data inside of your web apps? ECMAScript 6 has introduced some new and exciting ways to work with arrays and data collections, and we'll be covering what you need to know in this course. In this course, we'll walk through some hands-on examples of how to work with data collections, typed arrays, and advanced array methods using ECMAScript 6. Some of the major topics that we'll include in this course are making arrays more functional with some advanced methods, working with new data collections introduced in ECMAScript 6, which include sets that give you the ability to store unique values and maps that store data using key and value pairs, and finally, typed arrays, which allow you to work directly with binary data for faster performance and processing. By the end of this course, you'll know how to work with arrays, typed arrays, and how to store unique data with sets and key value pair data using maps. Before beginning this course, you should be familiar with the basics of JavaScript, including working with arrays. I hope you'll join me in this journey to learn array and collections with this JavaScript Arrays and Collections course, at Pluralsight.

Methods for Working with Arrays

Module Introduction

Hey everyone. My name is Jeff Batt, and welcome to this course on JavaScript Arrays and Collections. Now, we're going to be covering different kinds of methods that you can use when you're working with arrays and different kinds of collections like sets and maps, and also we're going to be talking about typed arrays. So really, we're going to dive into these different types of data storage where we can go in and store some of the data that we're working with with our different applications. So let me go through, before we dive into this course, and let me talk about what is not covered and what is covered in this course. And so what is not covered is the array basics. You should be coming into this course basically understanding the basics of arrays and how to create an array and how to add things to the arrays, how basic properties of the arrays, as



well as push and pop, being able to add things and and take things away from an array, as well as being able to access the basic arrays using dot notation or whatever you need to to access the basic parts of an array as well. You should be familiar with that, as well as for loops. You should be familiar with for loops and how to iterate over an array using a for loop. Now what is covered in this course is we'll be talking about advanced methods that you may have not actually come across yet, like list of, find, fill, and other types of methods like that. We'll talk about new ways to iterate through an array. We'll talk about for each. We'll also talk about for of and being able to iterate in those different methods, which actually make it a whole lot more simple to iterate through an array. We'll talk about these new data collections, like sets and maps and what are they and how you can store information inside of sets and access information from the sets, as well as access information from the map using keys and values. So really, we're going to dive into that in a certain section as well. Now we're also going to be talking about specific methods for those sets and maps, being able to access and being able to use the values or the keys in other types of methods like that. And then finally, we're going to wrap up this course talking about typed arrays. Typed arrays have very specific use cases, and they allow you to access binary data and get to that data if you're using APIs like Canvas or WebGL or anything like that. So we'll talk about what typed arrays are, how to create typed arrays, and how to access information from the typed arrays as well. So that's really what we're going to be covering in this course. It's really talking about data, so we're going to be using JavaScript to create these different types of sets and maps. We're going to be using these more advanced methods in arrays like list of and find and other things like that. If you're ready and this sounds like a good course for you, let's go ahead and dive in. But if you're not ready and you need to cover some of the basics of arrays, I highly recommend you go in and make sure that you check out the JavaScript Fundamentals course before coming here. Alright, let's go ahead and get started.

Reviewing the Sample Project

Before we dive in and start talking about using some of the advanced methods with arrays, let's go ahead and review the sample project that we're going to be working with. Now, this sample project is essentially a sales tracker. And so we have our Carved Rock Fitness, and whenever we get a new sale we're going to enter that sale into here. So we see our yearly revenue. We also see our last three months' revenue as well. We see our different department sales and how they're doing on the different department sales. And then we see the recent sales as well. Most of the course we'll be working with this monthly sales and we'll be adding the different sales in there, and we're going to be generating this chart and using the chart to really graph the sales that come in. We're going to have this modal here, and if I click on this modal, and I can enter in information about the sale, and that will generate the new data inside of this project. So to generate



the charts, we're actually going to be reviewing a framework called Chart.js, and we're going to dive into that in the next video. It's not complex to use, but we'll make sure that we know how to use it. And then we're going to use that to actually display the data, and the data is going to come from our arrays. It's also going to come from our sets. It's also going to come from our maps. So we're going to work with the different data formats and show you how to actually go in and create the data inside of those different formats and then access the data and use it to generate these different graphs. Now in the next video, you'll download a starter project and you'll download a finished project. The finished project is the project that we use throughout the entire course. And so if you want to go in and take a look at the code to see what was used throughout and you want to see the final vision, you can go ahead and take a look at that. That will be the finished project for this section, and then we'll have a finished project for the next section, and a finished project for the last section as well. Now, a lot of the layout has actually been built using Bootstrap. So we've attached some of the Bootstrap files so we can use the different layouts. That's how we get the responsiveness as well. That's also how we get some of the functionality like the modals. And we have different rows. We also have different columns that we can use in order to create these different types of layouts. And so you can see here, you can go through, it's marked up, you can see where the recent sales comes in, you can see the New Sale Entry, which is a modal, and that pops up with a new sale that allows me to actually enter in that information and go from there. So, we're not going to be working a lot with the HTML, and we're not going to be working a lot with the pages.js. But if you wanted to go in and customize the style, you're more than welcome to go in and do that. We are going to be working with this page.js. This is really where we're going to be putting in all of our code and working with the code and building the Chart.js and everything like that is going to be inside of this page.js. All right, so let's go ahead and dive into Chart.js and make sure we know how to build these charts using Chart.js.

Quick Intro to Chart.js

So now we're going to take a look into a framework that allows us to create charts easily called Chart.js. Now, we're not going to dive into everything that you can do with Chart.js, but we're at least going to use it to update and visually display our data. So let's go ahead and take a look at the Chart.js code and what we can do with that. Now, download the project files, the Module 2 Project Files, and start out with the 01 Starter Project. If you wanted to take a look at the finished project, you can go ahead and do that as well. But just to play it safe, I like to go ahead and copy and paste that onto my desktop and then close that out and start working with this. Let's go ahead and open this up in our code here. So I'm using Sublime Text. You can use Visual Studio Code, you can use Atom, whatever you wanted to. This is essentially what we're going to be working with. So I'm going to go ahead and actually open this up as a preview as well. So I'm double-



clicking on the folder and I'm going to go ahead and double-click on the index.html. And right now we don't have any data in there, we don't have any graph or anything like that. So what we want to do is we actually want to build the graphs, and I'm going to come down here into the index.html, and the first thing I'm going to do is go ahead and create our monthly sales. This is going to be a bar chart. So let's go ahead and just type in canvas, and now we're going to say id=monthlySales. That is going to be our ID, and that's how we communicate to it in the JavaScript. And we need to have two more attributes, which are going to be the width and it's also going to be the height. And the height is going to be 300, the width is going to be 400. All right, so let's go ahead and copy this, and this is going to be the same thing for our pie chart, but we need to change the ID to something else. I'm going to go ahead and just say deptSales, department sales. All right, so let's go ahead and open up our page.js, and we need to reference those two different objects using the ID. Now this is a little bit different than normal. We need to add on some additional context when we reference this. And so let's go ahead and just say, this is our Charts, these are our charts here, and we're going to say let ctx = document, let me move this out of the way so you can see it better, so document.getElementById, and then we're going to say inside of there we're going to be looking for our monthly sales, which I can come into the index.html, copy where it says monthlySales, go ahead and paste it right there. And so let's just end that with a semicolon. I'm going to copy that and paste it down one line. We're also going to do this for our pie chart. So let's go ahead and say pieCtx here, and instead of monthlySales, we're going to go back into our index.html, we're going to grab the departmentSales, come back into page.js there, and then just paste that there as well. Now, that's how you would get it normally with an ID; however, we need to do something additional when it comes to Chart.js. We actually need to say getContext, and then within there we need to actually get the 2d context. And so we're working with the 2D element. Because with Chart.js you can do a 3D, you can do 2D as well, so we need to make sure that we define it as the 2D. So we have those two things that we can communicate to. So from here, we essentially just need to actually go in and build out our chart. Now for the sake of time because we're not focused on Chart.js in this course, I'm just going to go ahead and copy that, and then I'm going to come in here on line 5 and I'm going to paste that. And I'm going to make sure that this is my bar chart and this is my monthlySales, and the monthlySales, let's actually do monthlySalesChart here, and that is going to create a new chart and it's going to give it the type of bar. And then it's also going to have labels that says Oct, Nov, and Dec, and then our data, which is going to be 12, 19, and 3. Now this will associate the Oct with 12, Nov with 19, and Dec with 3 automatically. It will also add on the background color. This is how we define the color with RGBA. Also the border width, right now we're defining it as 0. And there's a couple different options that you can do with the bar chart. And right now we're just starting out at 0. That's where the bar chart itself is going to start out. So if I went ahead and saved that and came in here and previewed it, you can see that the bar chart is right there, and all I need to do to update that is actually come in here



and change this to a different data point, and you can see it automatically adjusts. Really, we're just creating and we're feeding an array into this bar chart in order to create that data. I'm going to come in here and actually copy the pie chart here, and you can see the difference between the pie chart and the bar chart. Let's come down here and just call this one Pie and then just paste that in right there. And you can see we're just creating a variable. And the reason why I'm using `var` instead of `let` is because that is the way that it is defined in Chart.js and their documentation. So I'm just sticking with their documentation there, but everywhere else I'm using `let` here. All right, so here we're also passing in labels, and we're passing in the data as well. If I save that and hit Preview, you can see that I'm pulling up my pie chart here and I'm pulling up my bar chart. That's essentially what you need to do in order to work with Chart.js and to feed the data. We're going to be working with different arrays and methods in order to update the labels and update the data. So we'll be coming back to these data points often throughout the course. In the next video, let's talk about creating an array through a list of arguments

Create an Array from a List of Arguments

So let's talk about creating an array from a list of arguments. And so this is a method that we can use in order to actually create this array. So what we're going to do is come up here and we're going to just create an array here. So I'm going to say `let` and then say `monthlySales`, and this is going to be our monthly sales that contains all of the data for it. And then I'm going to create an array here by simply just saying `array.of`. This is a method that we're calling that will take the list of arguments and it will convert it into an array. So I'm going to give it three different arguments. I'm going to say 12, 9, and then I'm going to say 3 there. And that's essentially it. I've created this array just by simply calling a method and passing in some parameters. If you were to create an array just by saying `array`, for example, you'd still have to create it in a variable, but if you were to say `array` and then open close parentheses and then just say 5, that is actually not going to create an array with the 5 parameter, with the data 5 in there, it's going to create an array with five different spots. That's what we're avoiding here is by using the `of` method, we're creating an array with three different spots, and the first value is going to be 12, the second value is going to be 9, and the third value is going to be 3. So that is how you go in and pass in the parameters instead of just creating how many spots there. Coming down here, all we need to do to update the monthly sales is copy that and come down to where it says data right here, and I can just basically get rid of that array of data right there and copy and paste that in there. And now if I was to go ahead and preview this, you can see that it's going to use my 12, 9, and 3 array in my monthly sales instead of using the array that was there. That way I can have it be more dynamic and I can bring in the data from a form or something like that and then have it update automatically from there. I'm going to go ahead and do the same thing with the label. So I'm going to



say `monthlyLabels`, and I'm going to say `= Array.of`, and now I'm going to pass in strings instead of numbers. And so the first one is going to be `Oct`, the second one is going to be `Nov`, and then the third one is going to be `Dec`. So we're passing in three different strings there. And now I'm going to take this `monthlyLabels`, I'm going to copy that, get rid of this array where it says `labels` right here, and I'm going to go ahead and paste that there. Now if I was to go ahead and preview this, you can see that it has `Oct`, `Nov`, and `Dec`, which is what it was before, but now it's actually being pulled from this array instead. Let's do one more thing, I'm going to make sure that these are together and I'm going to create two more arrays here. The first one is going to be `department sales`, so we're going to `deptSales =`, and then `Array.of`, and then I'm going to say `12, 9, 3`, and then I'm going to say `let deptLabels = Array.of`, and then we'll pass in, we're going to have different types of categories, so we'll say `Hiking`, we'll also say `Running`, and then finally we'll say `Hunting`. All right, now if I basically just copy that and come down here and then let's update this variable to `Chart` just so we can talk to the chart here, and we're going to say the `labels` is, let's grab the `labels` right there, the `deptLabels`. I want to go ahead and update this right here. And then I'm going to go ahead and grab the `sales` and then I'm going to update this right here. And that's it. So let's go ahead and hit `Refresh` now. So you can see now our pie chart is updating, our bar chart is updating as well, and we've been able to create the arrays pretty quickly just by passing in the parameters

Using the Spread Operator with Arrays

The next thing that we're going to be talking about is the spread operator with arrays. Now, this comes in handy tremendously. Before I show you how to use it, let's go ahead and actually create a function that will add the yearly total. So I'm going to say `function`, and then we're going to say `addYearlyTotal`, and then I'm going to allow three different parameters. And so I'm going to say `a,b,c`. I'm going to make this more useful as the course goes on here, but let's just go ahead and accept those, and then I'm going to return `a+b+c`. I'm going to take all these three parameters, I'm going to add them together, and that's what it's going to return. So let's see how this is working. So I'm going to go ahead and just create a variable here, and I'm going to say `let yearlyTotal =`, and then we're going to say `addYearlyTotal`, we're going to call that function, and so I'm just going to go ahead and pass in those `monthlySales` there. If I was to go ahead and alert this, we'll have a problem here, and let's go ahead and see what that problem is. So I'm going to say `yearlyTotal`, and let's go back in and preview this, hit `Refresh`, and you'll notice that it has `12,9,3undefinedundefined`. The problem is it's not separating out these as parameters. It's not taking, okay, the first one, and that's going to be the first parameter, then the second one, that's going to be the second parameter, and so forth. We actually have to define it as separate parameters, and so I would have to come in here and actually say, well, let's pass in the first one, and then let's pass in the second one, and pass in the third one,



and so forth. So there's an easier way to do that. That's where spread operators comes into play. All you have to do to use a spread operator is if you have this array, you just add a dot, dot, dot when you're passing that in. And now it will take each of these individual items and it'll pass it in as separate parameters, and that's it. So if I go ahead and come back in and preview this, let's hit Refresh, you'll notice that it has 24 now, which is the total of 12, 9, and 3. So it just automatically does that for me, grabs them in and uses them as each individual parameters. Well, let's go ahead and actually update our total. Instead of alerting this, let's actually go in and grab our label. So I'm going to come up here where it says Yearly Revenue. I have this yearlyTotal ID, so I'm going to come back into page.js and I'm going to go ahead and call this yearlyTotalLabel. Let's actually just say yearlyLabel here and just say `= document.getElementById`, and we're going to go ahead and get that element, which is yearlyTotal. That's the ID inside of the HTML. So after that has grabbed and it's done the addition, we're going to go ahead and say `yearlyLabel.innerHTML`, and then we're going to say `equal`, and then we're going to concatenate a dollar sign on there. And then after that, we'll go ahead and add on the yearlyTotal. So it's going to update our label here. As soon as we hit Refresh, you'll notice there it is, it has 24 and it's automatically updated for me. So that's how we can go in and update that label and use the spread operator. So let's go ahead and actually update our monthly sales to be a lot more than just these small numbers. I want to go into the thousands here, so I'm going to say 12,000, 9000, and then let's go ahead and say 3000. Now if I come back in here, hit Refresh, you'll notice that the total has gone up a lot more. Now I'm going to come in and do a couple other things, and so I'm going to, down here, so let's go ahead and add on to this a little bit more. So I'm going to go ahead and create three different variables, and I'm going to say `octNums`, and then I'm going to pass in an `Array.of`, and then let's go ahead and just update random numbers here, 12,000, 1000, and then let's do 9000 there. And now I'm going to go ahead and copy that because I want to have three different ones. The first one is going to be October, the second one is going to be November, and then the third one is going to be December. And let's go ahead and update these to be random numbers, and I just want to have different numbers to work with here. So I'm going to go ahead and say that one is going to be 4000, let's make sure we have 4000 there. All right. I want to just go in and calculate all of these different totals. These are all my different parameters that I'm passing in, but I want to take all of those parameters and calculate them into one total. So I'm going to say `let total =`, and then we're going to say `Array.of`, and then we're going to pass in each of those numbers. Now, I don't have to actually pass in each individual one. I could actually use the spread operator to pass in each of those. So I can say `dot, dot, dot`, copy October, and then say `dot, dot, dot` again, go ahead and copy November, and then say `dot, dot, dot` again and then copy in December. Now that's going to basically create all of those. And if I wanted to, let's go ahead and `alert(total)`, and now if I hit Save and preview, you can see it does alert all those different numbers, but it doesn't actually add them together. So if I wanted to add them together, I would have to have each of them actually go



through the `addYearlyTotal`. So I can go ahead and just pass in each of those and then just make sure I have it on each of those there. And they're going inside of the parentheses there. And there we go. So now if I come back in here and hit Refresh, now I have three numbers. So now we have those three numbers, but let's go ahead and say `addTotal`, but let's actually come in here, well, let's do a spread operator here and then also `addTotal` to that spread operator. Do one more thing here. Hit Refresh, and now you can see I get my total. So I've been able to use the spread operator to get all of these numbers pretty quickly without having to type in each one or reference each location of each one as well. It's a pretty quick and easy way to be able to go in and to update that. Now if I wanted to also come in here and add other ones, I don't have to stick with just the spread operator. I can add other ones and use the spread operator as one of the parameters there. So that's an introduction to the spread operator. Let's go ahead and move on to the next section.

Using `Array.find` and `findIndex` to Find a Value

So we're going to use a method called `Array.find` and `findIndex` to find a value. So let's go ahead and come in here. And what we want to do is we want to set up a button first of all. If I come back into my example, you can see that we have this Add Sale, but I want to have a button that says Find \$1000. I want to find the first one that's over 1000. Let me go ahead and make sure that the first one here, and we're going to loop through all of those. But let me come in here and on the `monthlySales`, let me make sure that the first one is under 1000, and so we know that we're finding the one after that. I'm going to come back into the code here, and I'm going to go into the `index.html`, and I'm going to copy this Add button, and I'm going to go ahead and paste that. Let's go ahead and change this Add Sale over to Find Over \$1000. So that's what that button is going to do when it's going to find the first sale over 1000. Now I'm going to go ahead and add on an `onclick`. And in the `onclick`, we're going to call a function called `findOver1000` and then open and close parentheses, and we need to go ahead and create that function. So I'm going to come back into `page.js`. I don't want to use that `alert` anymore, so I'm going to go ahead and get rid of that `alert`. And then we're going to come down here and then just call that function and make sure that we come back into the `index.html`. I'm going to copy that function, come back into `page.js`, and then just paste that function there and then open close curly brackets, and there is our function. So now let's come in here and find the first value that's over 1000. I'm going to store this inside of a variable, and I'm going to say `firstThousand`. And now I'm going to say `equals`, and what we're going to do is we're going to search the monthly sales. So I'm going to say `monthlySales.find`, and we're going to look for an element. We're going to return an element that is greater than 1000. That's really what it's going to go in, it's going to go through and find an element that is greater than 1000, that element being a value essentially that it's going to do. Now I'm going to come in here and `alert`, and



we're going to say `alert firstThousand` and then end that with a semicolon. Now let's go ahead and preview this. You can see where it says `Add Sale`, and then we have the `Find Over $1000`. Let's go ahead and click on that. You can see that the first value is 9000. Even though the first value is not actually 9000, it's 500, but we're looking for that specific one that's over 1000. So if you're wanting to filter out anything that's above 500 or so forth, you can do a method like this. So the next thing that we're going to do is we're actually going to come in here and find index. So instead of `find`, let's say we wanted to get the actual index number. So remember that in an array, the index starts out with 0 and then goes to 1, 2, 3, 4, and 5. So if you need to get the index for any reason, you can use, instead of `find`, you can actually add on `index`, and then it will give you that number back. So if I come in here and hit `Refresh` again and then click on `Find Over $1000`, it's going to give me back the number 1, keeping in mind that the first one is 0, the second one is 1. All right, so that is how you actually go in and use the `find` and `findIndex` to find a number or a location of an element.

Using Array Fill

So let's say you wanted to take an array and then just put all the numbers back to zero, for example. Then you can use something like an array fill method, and we're going to actually cover that in this video. But before we do, we have a problem. If I come back into my example and I hit `Refresh` here. If I click on `Find Over $1000`, it does give me that alert. But if I click on `Close`, it's going to pop open that modal. So I'm going to go ahead and just get rid of that modal. And the way that I do that is by coming back into the HTML. I'm going to get rid of where it says `data-toggle` all the way down to where it says `exampleModal`. That will get rid of it. That's a bootstrap thing that will just make sure that that doesn't pop up. All right, so let's go ahead, and while we're here, we're going to copy this and we're going to go ahead and paste that, and we're going to change the label here to `Reset to 0`. It's going to take that number, and it's going to reset it back to 0. And let's go ahead and call the function of `resetNumbers` or just `resetNum` there. And so I'm going to go ahead and copy that function, come into `page.js`, and let's go ahead and create the function of `resetNumber`. Now it's going to go into the `monthlySales`, and it's going to essentially fill all of them back to 0. So I'm going to say `monthlySales.fill`, and then I'm going to pass in the value that all of them are going to be set at. So I'm going to go ahead and say 0 there. And now what I want to do is I actually want to update where we have `monthlySales` here. I want to update this chart. And the way that I could do that in `Chart.js` is just by copying the name of that variable there and then just pasting it and then just clicking update open close parentheses, and that will update the values, and it will reset it all back to 0 there. So let's go ahead and hit `Refresh`, and you can see that we have our numbers. But I have this `reset to 0`, and if I click on that, it takes all of them down to 0. So that's just a useful method to be able to say, okay, I want to start back from scratch. And then as you start to add more, then we'll start to add



more and start to fill out the chart and everything like that. So just a useful method that you may need to use to reset data or to set data across the board on an array.

Methods for Iterating through Arrays

Now we're going to be talking about different methods to be able to iterate through an array. We're going to focus more on `forEach` in this video. And in the next section, we're going to be talking about `forEachOf`. And these are new methods to be able to go through each of the different elements inside of the array and work with that array. So let's go ahead and come in here. And really, we have some inefficient code. We have this `a`, `b`, `c`, and its returning the value. But if we had additional parameters like this one right here, it's going to throw off that code. It's not going to work. And so what we want to do is we want to modify this code a little bit. And so I'm going to come in here, and I'm going to just modify this to accept one parameter, and I'm going to go ahead and get rid of this `return`, and I'm going to get rid of this line right here or at least comment out the line, line 20, and I'm going to get rid of this `yearlyTotal` right there as well. I still want this `yearlyLabel`. That's going to go in and update the label, but we need to do some calculations with this before it actually gets updated. So we'll just leave that there. All right, so coming in here, let's create a variable first of all. So I'm going to say let `yearlyTotal`. And by default, it's going to be 0. All right, so now I'm going to come in here and I'm going to say `yearlyTotal` is equal to, every time this function gets called, it's equal to whatever the parameter is that's getting past in plus the current amount of `yearlyTotal`, whatever amount it may be at that time. And that's really all that I want to do inside of that function. Now what I want to do is I actually want to loop through this `monthlySales` array, and I actually want to pass in whatever the parameter is right here, I want to go ahead and pass that in. Now I can simply just go ahead and just type in or paste in the name of the array and then just do a `forEach` loop across all of that. And that's how I do a `forEach`, just `for` and then a capital `E` for `forEach` there. And now if I wanted to call a function for each of these different parameters, all I have to do is copy in that's function name into the parentheses there. And what it will do is it will basically go through this function, and it will take the `yearlyTotal`, add on whatever the parameter is to the `yearlyTotal`, and then it's going to go ahead and update our total there. And since we have this label right here, after all that calculation is done, it's going to go ahead and update the `innerHTML` to that `yearlyTotal`. That's it. So now I can come in here and hit `Refresh`, and you'll notice that it automatically updates for me. So I can see here that it has \$12,500. If I came back into my code here and I went ahead and updated that amount to say, okay, let's pass in another parameter like 400 here, hit `Refresh`, you can see that it automatically updates. So it's just a more dynamic way to be able to handle any amount that this array may have. And so it'll just go through each of those arrays, and it will go ahead and add that to our total. In the next section, we're going to be talking



about the `forOf` loop and how to use the `forOf` loop in sets, but it's going to work exactly the same way inside of arrays. So we'll focus on that in the next section. But let's go ahead and talk about what we've learned about in this section and wrap everything up before we move on to the next section.

Module Conclusion

So we have covered a lot in this section, so let's just quickly review what we've learned here. We've talked about the course overview, and we talked about what we'll be learning in each of the different sections and what we won't be learning and some of the different basics of arrays that you may need to go back and review from a different course. We also talked about `Chart.js` and how to update `Chart.js`. We talked about how to add in the different values of `Chart.js`, as well. We talked about `Array.of`, being able to create an array from a list of different values. We talked about using the spread operator as a quick way to be able to add all the different values of an array inside of parameter or something like that. We also talked about `Array.find`, as well as `Array.findIndex` if you wanted to find a specific index. We talked about using `Array.fill` to be able to fill out the content of the array. Like I mentioned before, that's mostly if you wanted to reset it all to a certain number. And then finally, we talked about iterating through an array and using the `for each` to be able to go through and quickly take all the values and use all the values to do something more efficiently. So, that's really what we've covered. We talked about more advanced features for these standard arrays, but now we're going to go in and dive into working with sets, which sets are data collections, and we can use sets for being able to handle data more efficiently and adding on some additional methods that we can do that's beyond our standard array methods that we have here. So really, we're going to dive into sets, you're going to become familiar with sets. In the next section and the section after that, we'll be talking about maps and how they differ and some of the different methods you can use in maps and then, finally, typed arrays. So, we're just scratching the surface. Let's go ahead and jump onto the next section.

Working with Sets

Module Introduction

Now we're going to get into different types of data collections that we can work with that goes beyond arrays, and the first set of data collection that we're going to be talking about is sets. So we're going to be talking about what sets are, how to actually work with sets, how to add content to sets, how to remove content from sets as well, how we can iterate through sets and really grab out the values



of the sets. So, that's really what we're going to be focused on in this section. Before we dive too deep into it, let's just talk about data collections as a whole in the next video.

Reviewing Data Collections

Before we dive into sets, let's just make sure that we understand what data collections are. Now data collections can be found in other languages. We have Python, which uses lists, as well as tuples, as well as dictionaries, and so it's just different ways to be able to store data and work with the data. Java uses lists as well, sets and maps similar to JavaScript that we're going to be learning about here in a little bit, as well as queues. So Ruby uses hashes and arrays. So data collections are really not anything that's new to programming languages. So we're going to be focused specifically on the new enhancements of JavaScript, and so that's ECMAScript 6, which was released the 17th of June in 2015, and it brought a whole lot of new features with it. And one of the main features that it brought was new data collections. Now a data collection stores and structures large amounts of data for easy access. So it's a way to organize the data and a way to be able to get that data and use that data, and it provides methods to be able to access and use that data. So before ECMAScript 6, the only thing that we could really use was arrays and objects inside of JavaScript. So ECMAScript 6 gave us Sets, as well as Maps, and then WeakSets and WeakMaps. So in this section, we're going to be talking about Sets and WeakSets. In the next section, we're going to be talking about Maps and WeakMaps and when to use Maps and when not to use Maps and when to use WeakMaps and when to use Sets and so forth. Before we dive into that though, there is a couple more things that I wanted to talk about, and that's really the data types that we can work with. And so there's really eight different types of data types that we can work with. There's booleans. There's nulls. There's undefined data. There's also number data. There's big integer data. There's strings, and then there's symbols, as well as objects. You can work with objects. Now seven of these data types are actually primitives, and that's important to know. The final one is object. Now primitives really can't be changed. You can't change what number 1 actually stands for, but objects can actually be changed. And so they're value in memory which can be referenced by an identifier. So an object itself you could have stored inside of a set, and you can reference that with the ID of that object. And we're going to be working with those different objects. We're going to be working with the different sets and these different types of data values. And so we're going to be working with primitives. We're also going to be working with objects and then storing each of those data types inside of our set. So let's go ahead and actually introduce sets, and let's talk about sets in the next video.



Introducing Sets

Before we get into sets and creating sets and adding sets and adding data and other things like that, I wanted to talk about what sets are in general. Now sets enable you to store unique values of any type, whether it's a primitive or whether it's an object. So all that primitive data that we talked about in the last video, we can store that type of data inside of a set. Or we can create an object and we can reference that inside of an object as well. So really, one thing that helps set stands out is this data type, although it could be similar to an array, must be unique values. And so that means if I have this set of data and I decide to enter content into that data, then I can say, okay, one of the pieces of the data is 52. Another piece is 183. Another piece is 200. Another piece is 52, and another piece is 47. Well, let's take a look at this because what happens is 52 is actually repeated. And so you'll notice that the first number is 52 as well. So what happens is if you start to iterate or you start to look for a value of 52, it'll go through all of these different values, but it will notice that this is a duplicate, and it will ignore that value and just move on to the next value. So it does need to be unique values. You can have duplicates in there, but it will completely ignore them and so it really doesn't do you any good. So if there are a set of numbers where you need to have these unique values, then sets are actually a good way to be able to store that data and to make sure that you do not have any duplicate numbers if that's what you need. But they can be of any type, and so we can have the numbers. We can have strings. We can have functions. You can actually have a function inside of the set that will call and return a value as well. You can have objects and then you can also have just, you know, whatever other big integers or any other type of number there, and all of that could be stored inside of the set. Now sets have their own properties and methods as well. The properties are a little bit more limited. Instead of a `array.length`, you actually have a `set.size`. And so you can determine the size of the set by using that property. Now the methods, it actually gives us a lot of methods to work with. And so we can add, we can clear out all the sets and so we don't have to set all the values or pop all the values or anything like that. We can just clear him out. We can delete certain data from the set. We can also reference the different entries in the set. We can iterate through the set using `forEach`. We can check to see if the set has a certain type of value just by using `has`. And then we can reference all of the keys in the set, and we can references all of the values in the set as well. So those are the different methods that we can use in order to work with these sets, and that's really the power of the sets is it gives us these additional methods, and it allows us to work with sets in more functional ways than we could with arrays. So let's talk about actually adding and removing values from a set. So let's get back into the code and start working with that.

Adding and Removing Values to a Set



We've talked about what a set is. We've also talked about different properties and different methods that you can use on a set. So make sure that you download the sample files for this module. Make sure that you also store the sample files from the last module. Make sure that you start with the 01 Starter Project file and copy that onto your desktop. I'm going to go ahead and double-click on that, and I'm going to double-click on the index.html here just to make sure that I have that up. And I'm also going to open this up inside of Sublime. Really what's changed is mostly the module that pops up. We have our month that is saving it inside of a monthId. We also have our itemAmount, which is going to save it inside of the ID called itemAmount. And then we're also calling a function called addSale. Open up page.js. We have our reference to that newAmount, so we can grab the value from that form. We also have the reference to the newMonth. And I've also commented out the chart, the pie chart as well, and then I've removed some of the code from the last section as well. So we're going to add some new code right here. The first thing that we're going to do is we're going to create a const, and then we're going to say, okay, this is going to be monthlySales. We're going to store our monthly sales inside of here. And this is going to equal a new Set. We're also going to do the same thing for our monthly labels. So we're going to go ahead and say this is also equal to a new Set. Now that we have that there, we want to add our content in as soon as that button is clicked. Now we're calling the function on that Add Sale button called addSale. So let's go ahead and create that function, and we're going to name it addSale. I copied and pasted that in there and then open close curly brackets there. We're going to say monthlySales is going to add a value, and that value is going to be our newAmount and then .value. Now keep in mind that this is actually going to be passed in as a string. We're going to convert it into a number later. But for now, just keep that in mind because when we alert it or when we try to find it, it's actually going to be a string and not a number. We're going to do the same thing here with labels. So we're going to go ahead and say newMonth, and that's going to be the value, so .value, and there we go. Now I'm going to alert a message letting them know how big the actual monthly sales is. And so I can use that by using the property of this set. So I can go ahead and say You have entered in, and then we're going to add on monthlySales.size, and then we'll add on one more thing. So it's going to say you have entered in number of sales, so two sales or three sales or whatever it is there. Let's go ahead and make sure that this is working in that the values are being entered in. So I'm going to come back in here and hit Refresh, and then I'm going to enter in the month, we'll enter in the value, and then we'll click on Add Sale. Now it says that I've entered in one sale, so let's go ahead and close that. Let's add it again. And let's say this is going to be November, and this is going to be 1500. If I click on Add Sale here, it now says that I've entered in two sales. Well, how do I get the value of those sales? Well I can come back in here and I can say console.log and then say monthlySales. That's going to alert the content of the monthly sales as soon as I add it. So let's go back in here and hit Refresh and click on Add Sale. And the first one is going to be October, the second one is going to be 1200. It's still going to



alert that message for me, but let's go ahead and right-click and go down to inspect element. And now if I come in here and click on this, then I go down to Set, and it says 1200 right there. So that's adding a new sale. But what if we wanted to delete something? Well, we can actually delete something by using the delete method. If I come back into my index.html, I'm going to go up to this button here that says Fill Sales, and we're going to change this to deleteVal. It's going to call a function called DeleteVal. I'm going to copy that, and so we're going to say Delete. Now I'm going to come back into my page.js, and let's go ahead and create that function, deleteVal open close parentheses, open close curly bracket. Now we're going to say monthlySales, we're going to .delete, and that is going to delete the value of 1500. Keep in mind, I'm putting this inside of a string because we have not converted it into a number. And so I'm really trying to find a string that is in the amount of 1500. But let's console.log the amount again. So let's say we're going to console.log on monthlySales just so it alerts it again there. So if I go ahead and hit Refresh, let's add our first one as 1500, and then let's add our second one as 1200. And we'll say the month is going to be November, and that is going to be 1200, and then click on Add Sale. If I right-click and go down to Inspect Element, it's going to have those two different values there. If expand this out, we have the 1500, and we have the 1200. If I click on Delete Sales, nothing really alerts there. But if I go into Inspect Element and then go into the console.log, it will only give me 1200, so that's how I get rid of that. Now there are a couple different ways to actually add content into this. I'm going to actually create just a test one here and say test = new Set. If I wanted to come in here and add several different values, I can actually chain this. And so I can say test.add, and let's go ahead and add five and then .add add 6 as well. And so that's one way to just chain the different values into adding to that one set. Or if I'm creating the set, I can actually create values right inside of here as well just by passing that inside of brackets, and that is going to be my initial values inside of there. So that's another way. But you can also pass in other values. You can pass in strings and then you can also pass in objects. And so if I wanted to go ahead and create like an object here, I just open close curly bracket there, and then I can give it a value of, let's say, test: and then give it a string of This. Now if I come in here and console.log, let's just say test there. If I'm going to add it inside of the set, I actually need to add it inside of brackets like that. Hit Refresh. And now if I right-click, go into Inspect Elements, you can see that it has now logged that object. So those are different ways to be able to add, to be able to delete, and to be able to work with sets. So in the next video, we're going to talk about iterating. So we're going to be talking about going through the different values and finding certain ones and updating content based off the values that we get here. So we'll talk about that in the next video.

Iterating a Set

So now we're going to be talking about iterating a set, how we actually go



through a set and we use the set values to do whatever we need to. So let's go ahead and come in here and clean up the code a little bit. I'm going to get rid of this test, and I'm also going to get rid of this alert and this console inside of `addSales`. So the first thing we want to do though is we want to use this `forOf` loop. We have already talked about the `forEach` loop, but we're going to use this `forOf` loop. So I'm going to say `for`, and I'm going to say `let`, and then we're going to say `total` of `monthlySales`. We're going to iterate through `monthlySales`. And then we're going to assign each of those to this `total` value. Now what I can do is I can actually open this up inside of curly brackets or, a shortcut, if I'm just going to do one line of code is I can say `console.log`, and then I can say `total`. This is going to go through each of them, and it's going to log the total. Let's go ahead and preview this. I'm going to hit Refresh and then click on Add Sale. Now I'll enter in the value of 1200. And at this point, it should only have one value. So if I click on Add Sale, it's going to `console.log` that one value, which is 1200. Well, I could do a whole lot more, but I don't want to do this on the Add. I actually want to do this on this Find Sale. Or this is really going to be our figure out our total. So let's go ahead and go into our `index.html`. Let's find that button where it says Find Sale. And instead of Find Over \$1000, we're going to say `addTotal`. And then we're going to change the label to `addTotal` here as well just so it makes more sense there. So now we need to create that function though, so I'm going to come back into `page.js`, and I'm going to come down below this `delete` value. I'm going to say function `addTotal` open close parentheses, open close curly bracket, and there we go. Now what we want to do inside of here is we want to do a `for` loop, and we're going to say `for` and then `let` the amount of `monthlySales`. And then inside of here, I want to go ahead and open and close curly bracket, and now I'm going to `alert`. Just to make sure this is working, we're going to `alert` the amount here. I'm going to save that and hit Refresh. Really, nothing should happen at this time. Let's go ahead and actually add in some values and then say `add value`, and then let's add one more and then 1500. Now if I click on that, it'll say 1200, and then it alerts to 1500, and it will loop through each of those. That way when I click on that, it's basically going to update this label. That's our end goal is to go in and update that label. So what we want to do is any time that this `addTotal` actually gets added here, what we want to do is initially set back the `yearlyTotal` to 0. If we don't do this, it's actually going to take the value, and then it's going to add onto that value. So here we're going to say `yearlyTotal` is equal to the amount plus `yearlyTotal`. So we're going to take whatever the current amount is as we're starting to loop, and we're going to add on the new amount here. Now I'm going to update that `yearlyLabel`. We have this `yearlyLabel` as well that's going to update the label for the element called `yearlyTotal`. So we're going to say `.innerHTML` is equal to `yearlyTotal`, and there we go. We have a problem though, so let me go ahead and preview this. Let's start to add in a value, and I'm going to say October. I'm going to add in 1200 for the first one, and then let's add in November, and this one is going to be 1500 and then click on Add Sale. Now if I click on Add Total, notice that 1500 gets added, 1200 gets added, but 0 gets added as well because what's happening is, remember, this is the issue that we



were running into in the last video is the value from that form is a string. It's not actually a number. So what happens when you add one string to another string? Well, that uses concatenation, so it actually takes that string and adds it to the other string. And really, we're just displaying a string at this point. We want to work with this as a value and not a string. So what we have to do is when we actually add this value to our monthly sales is we want to parse the integer. We want to grab this value from the form, and we actually want to parse it and create an integer. And so that's really what we do is we wrap the new amount into a value. Now if I hit Save and let's come back in here and hit Refresh and start to add in the values now, we'll go ahead and say October is going to be 1200 and then November is going to be 1500 and then click on Add Sale. Now if I click Add Total, you'll notice it does the math now. It is now 2700, and it automatically did that for me. So just by adding this `parseInt` method around that value, it took the string and it converted it into an integer. Now there is another way that we can actually go in and loop through our content here. Instead of the `forOf`, we can actually go in and use a `forEach`. And so if I came in here, and let's do this under the delete value. Let's actually come in here and say `monthlySales.forEach`. Now when you're using the `forEach`, you actually need to pass in three different values, the first value because it returns two different values back to you. So the first value is really what we need to use, and then the second value is kind of a duplicate from the first value. But the first one and the second one are pretty much the same. What I need to do now is say, okay, what am I actually going to be looping through? Well, that's going to be our `monthlySales`. This is going to return a value. So you do return by an equal sign and then a greater sign as well, and then we're going to open and close this with a curly bracket, and this is where we're going to place in our amount. So I'm going to go ahead and alert that, and I'm going to say `value1`, but it's going to loop through that and then give me the value. Now let's go ahead and just hit Refresh and then click on Add Sale. And now if I click October and then add in a value there, I go ahead and type in November, add a value there. Click on Add Sale. Now I'm going to hit Delete Sales, and you can see that alerted the 1200, and now it's alerted the 1500. It will loop through that using this `forEach` loop. So just keep that in mind that you can do it in either one of these ways. You could do the `forOf` or you could do the `forEach`, whatever way you're most comfortable with. Those are the different methods for being able to iterate your different sets.

Updating the Graphs

So we've been updating our content inside of sets, but now we actually need to take those values and we need to update the graphs in our charts here. So what we need to do is come in here, and we see that if I come down to the Add Sale right here, we're adding to our `monthlyLabels` this value. What I want to do is actually go in and take those values and actually convert it over to data that can actually be used inside of `Chart.js`. Let's go ahead and uncomment out this



monthly sales chart. But you can see here I'm referencing `monthlyLabels`. Well, I don't really have those labels inside of a variable. I do have that label as a set, but watch what happens. If I come in here and hit Refresh, you can see that it does not actually get updated. Just because we're using Chart.js, we actually need to take that, and we need to create it into an array here. What I'm going to do from here and probably the easiest way to do this is to actually come into the `addSale`, and what we're going to do is we're going to say `monthlySalesChart`, and this is a method for Chart.js. So we can say `.data.labels`, and then we're going to say `equals Array.from` and then open close parentheses, and then we're going to say this is going to be from our `monthlyLabels`. We've used `arrayOf`, but we have not used `arrayFrom`. So it's very similar. `ArrayFrom` will actually take these different values, and this is what we're going to create the array from. And so now we're basically telling Chart.js that we're going into the data and then we're going into the labels and we're updating it and saying this is now the new value. So we can actually get rid of that and just create a blank array for now because that's going to be updated later once we actually go through and add our sale. So I'm going to come in here and hit Refresh now. And you can see that we have no label, but our chart is actually showing up, which is a good sign. If I click on Add Sale, and then I go ahead and just say October, and now we're going to enter in the value of 1200. I'm going to click on Add Sale, and it's actually updated. However, it's not updating this chart. So what we need to do is one more thing. If we come back into here, after we make a change to the Chart.js, we actually need to call an update method. So I'm going to say `charts monthlySalesChart.update` open close parentheses. We're going to call in that method, and that's going to update the chart for us. So I'm going to come back in here and enter in the new labels. We'll say October 1200 and then go ahead and click on Add Sale. And you can see, it's automatically updated that label. So that's how you do the monthly labels. But let's actually go in and inside of this monthly amount here, so the let amount of `monthlySales`. This is how we get the total here. What we're going to do is we're actually going to create a new `forEach` loop that's inside of here. So let's go in and we have to loop through, we have to use this `forEach` method to loop through the different values and then update the values inside of our data here. So I'm going to say `monthlySalesChart`, and then I'm going to say `.data.datasets`. That's how we're getting down to the value. And then we're going to say `forEach`. We're going to use this `forEach`. And then inside of there, we're actually going to actually pass in the dataset value, open close curly brackets and bring that down a line. And now we're going to say `dataset.data.push`, and we're going to push in this amount. So I'm going to basically make sure that this starts out with no data there. This is going to be a blank array there. And because we're already getting the amount and we're looping that in, essentially, we're basically just going to loop through all the datasets inside of our data, and we're going to push in the amount that we get back there. But one thing that we forgot to do is actually make sure that we return the values. And so now if I hit Save and let's come back in here and hit Refresh, let's go ahead and add a value here, and we'll say October, and then



we'll do 1200 and click on Add Sale. So let's actually do this at the same time that we're updating, we're adding our sale here. So I'm actually going to take this `yearlyTotal` in this for loop, and I'm going to cut that. And so as soon as I click on Add Sale is when I want to update that. So I'm going to come in here and I'm going to past that there. And now I'm actually going to get rid of this because we're kind of doing the same thing right here, amounts of `monthlySales`. We don't need all of that really. So we're going to say `yearlyTotal`, and now we're going to loop through that. We're going to push in our data, and then we're going to update the labels right there. And then after all of that's done, we're going to update our sales chart. So let's go ahead and preview this now. And let's say October 1200 and then click on Add Sale. And you can see that it's updated the chart automatically with 1200. But watch this. If I go ahead and add in November, for example, and then I say this one is going to be 1400 and then click on Add Sale, you'll notice that November is actually registering as 1200 because it takes the values from initially when I added the first time, and it adds in both values the second time. So we actually have three values and you can't see the third one because it doesn't have a label. But we actually have three values that are inside of here. So I actually need to reset the data and clear out the data before I actually start adding anything inside of there. Exactly how I deal with this `yearlyTotal` is I set it back to 0 before we loop through everything. And the way that you do that is actually by doing a `forEach` loop as well. So I need to come in here right after `yearlyTotal`, and then I need to actually loop through the dataset, exactly what we've done down here. And I need to set the dataset data back to this empty array. Just by doing that, now if I come in here and click on Add Sale and if I come into October and then say 1200 then click on Add Sell, it'll add that, but as soon as I start to add other ones here, so now I'm going to say this one is going to be 1400 and click on Add Sale, it adds the second one as 1400 and the first one as 1200. So it essentially just clears out the array, and then it starts the array again as it starts to iterate through the set values. So that's really how you go in and you update the graphs. So you go in and you iterate through the different sets, and then you change the iteration back into an array and then update the `Chart.js`. And the only reason why I have to do that is because I'm working with `Chart.js`. If `Chart.js` updates where it will actually accept values from a set or a map, then you won't have to do this. You won't have to convert it back into an array. So in the next video, we're actually going to be talking about weak sets and what weak sets are and how you actually start to work with weak sets. And there is a little bit of a difference between sets and weak sets, and so we'll go ahead and talk about what that difference is and then dive in to working with weak sets.

Differences between a Set and WeakSet

So let's explore some of the differences between a set and a weak set. So first of all, it can only contain objects. It cannot contain any type of primitive data



types, and the objects themselves are only referenced weakly, and we'll talk about what that is here in a second. It cannot be iterable so I cannot reiterate through a weak set. It's only really designed to contain a certain type of information. It has no access to the size property. The reason why is because these references are contained weakly, which means that there may not be a reference or you may get rid of that reference or something like that, and so that's one of the reasons why you cannot use the size property. Really it comes down to this is being garbage collected. You're opening up certain parts of your data to allow it to be garbage collected. Now you may be wondering, okay, Jeff, what is garbage collection? So basically garbage collection happens by default inside of browsers, and it monitors the memory allocation, and it determines when a block of allocated memory is no longer needed, and it basically reclaims it. Sets, by default, will not get rid of that memory. So it will just stay there in that memory. When you decide inside of a set that's saying, hey, I no longer need this, you actually would want to use a weak set instead of a set because what that does is when you define it as a weak set, it says that if this is no longer being used, it can be garbage collected. You can get rid of it, no longer hold it inside of memory. And that's really the power of being able to use these weak sets. So sets you have access to all of these methods, but with weak sets, you only have add, you have delete, you have has as well, so you can see if there's a certain value or something like that. But that's really it. You only have those references, so it's really just if you wanted to store certain types of data that you're okay with it being garbage collected, you're okay if the data is not there, you're basically just holding it as a certain type of value if it is there, and you can store it inside of a weak set. So in the next video, we're actually going to go in and start working with a weak set right instead of our code.

Working with WeakSets

Okay, now we're going to be talking about working with weak sets. So let's go into our code, and I'm going to go ahead and create a new set. This one is going to be a weak set. We're going to call this categories. Now I don't want to actually store a category that's not chosen because that will just take up unnecessary space. So this is a good use case for being able to use a weak set instead of using a set. So I'm going to say new WeakSet in order to create that new weak set. Now I can simply go in here and start adding a category. So I could come in and say categories, and then I'm going to do .add, and so we have the add method that we can add for here. But notice if I come in here and just say Hiking, for example, and then I do a console or let's do an alert on categories, and so I'm going to say categories here, we're going to run into a problem. So I'm going to save that, and I'm going to go ahead and preview this. And if I hit right-click here, I have a problem there because that is a non object key. Remember that we cannot work with primitives. We cannot use strings. We cannot use numbers or anything like that. We can only really use objects. And so what I would need to do is actually



add this as an object. And when you create an object, you have to put it inside of curly brackets and then you have to give it a name. So I'm going to say category and then colon, and then I'm going to go ahead and put Hiking there now, So I'm going to say Hiking, and now let's go ahead and try that. Hit Refresh. So you can see that it alerts, but it says that it's an object. We actually need to get to that in a different way. So we'll work with that here in a second. One way to work with objects a little bit easier is I can actually store that object inside of a variable. So let's just say let hiking =, and then we're going to say category: and then Hiking there. Now when I'm adding it, I can actually add Hiking right there, and that will be a little bit easier. So now if I wanted to say I wanted to check to see if hiking is there, all I have to do in the alert is say categories.has and then hiking. Does this have hiking? And it will alert if that's true or not. So if I go ahead and hit Refresh, it's going to alert that, yes, it's true. It does have hiking. So if you just store it inside of a variable there, it's a little bit easier to add and actually check to see if it has a certain value as well as you're going through the code here. Now I'm going to go ahead and create these three different values because we have in our form if I come back into my preview here, if I click on Add Sale, we have Hiking, Running, and Hunting. So I want to store each of those inside of a variable here. So we already have Hiking, and I'm going to copy that and just paste that here. I'll change this label to Running, and I will change this label to Hunting. Okay, so we have those different categories that we can go ahead and add. But I want to grab the value from each of those radio elements. If I go back into the index.html, I want to grab to see if this is checked or not. So you can see we have Hiking as label1, we have Running as label2, and we have Hunting as label3. So I'm going to go ahead and just store each of those labels, those IDs inside of a variable here. So I'm going to copy and paste this, and you're more than welcome to pause and write this out. So now what I need to do is under addSale, I'm going to come in and check to see if those labels have been added. Coming in here, let's go ahead and just say if hikingRadio, and then we're going to say .checked. That's how you check to see if a radio button has been selected. If that has been checked, then we're going to do something here, and that's where we're going to say categories. We're going to get rid of our categories.add up here. Let's get rid of that alert as well. And if that's been checked, we're going to add categories.add, and then we're going to say hiking. Now the nice thing about the weak set is if this never gets run, that is actually even nowhere to be found inside of the memory. And so it's being garbage collected. It's not being used at all. If I remove this, it will be garbage collected at the next available time. And so again, that's one of the benefits to this. Now we can also say else if, and then we can check to see if the runningRadio, let's scroll up here, runningRadio right there .checked, and then we can go ahead and add that running to our category as well. So I'm just going to copy in the rest of this here. In fact, I'll just paste it right there. So else if runningRadio is checked, then we're going to go ahead and add categories of running. If this is hunting is checked, we're going to go add that to the category there as well. And I'm just going to console.log, and we're going to do categories and see what we get



there. Let's go ahead and preview this. Let's hit Add Sale, October 1200, and then let's go ahead and add Hiking there. Now if I right-click here, go into my console, there's my weak set, and you can see that it's added the category of hiking. So just a nice way to be able to add the values when you need it, to not use it when you don't need it. It's not something that I'm going to need to iterate over. It's just basically going to be inside of my category, inside of my value there. So that's how you work with the weak set inside of JavaScript. In the next video, we're going to talk about what we've learned in this section. And then in the next section, we're going to be talking about maps.

Module Conclusion

So we've learned a lot in this section, and I wanted to review what we've learned before we move on to maps. So here's what we've really been able to dive into. We initially started talking about data collections and what data collections are and how they're new to ECMAScript 6. We then went into sets and talked about what sets are, how you can create sets, as well as adding and removing different values from those sets and then how to iterate through the sets to grab the values and to work with those values in whatever way that you want. We then talked about the differences between sets and weak sets and when you would want to use a set, when you would want to use a weak set. And then finally, we wrapped up with actually working with weak sets and using that data inside of our code. And that's initially what you'd want to know when you start working with sets. We're going to talk about actually using maps in the next section. Now maps gives us some additional functionality, and one of the nice things is it allows us to work with keys and values. And so we can define a key in any way that we want, and we can define a value in any way that we want as well. So let's go ahead and dive into maps in the next section.

Creating and Using Maps

Module Introduction

We've learned about advanced arrays. We've also learned about sets and what you can do with sets. And in this section, we're really going to dive into maps. We're going to talk about what the difference is between the sets and the maps. We're also going to be talking about weak maps, similar to what we talked about with weak sets as well. Maps add some additional functionality on top of sets. It is a very similar way to be able to store the data, but to be able to access the data and how you store the data is different. So let's go ahead and dive in and start talking about what maps are to begin with.



Introducing Maps

Before we dive in and start using maps, let's go ahead and just talk about maps in general so we understand what maps are and some of the differences with maps compared to sets. Now maps use a key-value pair, and it keeps the original insertion order. That's important to know. We're going to be talking about that here in a second. But it keeps it in order. And any value, like objects or primitive values, can actually be used as either the key or the value. With maps, you have to define a key, and you have to define a value. Just like with sets, when we added a value to a set, we had to pass in the value of that set. But instead with map, we actually have to pass in the key and the value. And there's important things to notate here. Notice that instead of add, we're using set. So we're setting the value, and we're setting the value of the key, and we're setting the value of that key as well. It's really important to know because when we want to reference this map as well, when we want to grab the key, we need to know what that key is to be able to identify it. So really, maps are a storage location just like sets are, but we're storing different types of values in this key-value pair. We can have strings as part of the key. We can have numbers as part of the key. We can even have functions as part of the key as well. And we can have objects as part of the key as well. It just depends on what you want to use and how you want the maps to function. An important thing to notate because you can do something similar with keys and values with arrays, but the problem with the arrays is if you try to iterate in order, there's no guarantee that it will actually go in the order that you've inserted it. If you've inserted it and you wanted to go through that specific order, you can't guarantee that. It'll jump around sometimes. Sometimes it will get in that order. Sometimes it may not. There's just no guarantee. But with maps, there is a guarantee, and it will go in that insertion order. It'll go one by one and go through all of your different values that you have entered in. Let's talk about the different properties and methods that come with maps. Some of them are very similar to sets, but other ones are different. So size is the same as the set. So we have access to the size, but that's pretty much it. We can get the size of the maps, but we don't have access to a lot of other properties there. But as far as methods go, we have clear so we can clear out the map completely. We have delete so we can delete specific items of the map as well. We have entries so we can get into the entries of the map. We have forEach to be able to iterate through them. We have get so we can actually get the value. We also have set so we can set the value as well. We can check to see if a map has a certain value. We can also access the keys. If we want to just get all the keys for all the entries, we can go in and just access the keys. And we can access the values. That's the property that we're going to be working with throughout this section and the different methods that we're going to be working with throughout this section. So I just wanted to make sure that we were clear on the different methods available when you're working with maps.



Adding and Removing Values in a Map

Now let's talk about creating a map and then adding and removing values from that map as well. I'm going to come into this project and make sure you download the project files. Save the project files that you have from the set section and make sure that you have that so you can go back in and reference. But I'm going to download the project files for this section, and I've modified the code a little bit so you can just start off from where I'm starting off. I'm going to go ahead and copy this starter project, and then I'm going to paste it on my desktop. Let's go ahead and preview this first of all. So I'm going to double-click on that folder and then double-click on index.html. So nothing's really set up to work on this yet. I'm going to go ahead and open this up in Sublime, and I'm going to open up the index.html. It's pretty much the same here. There are a couple differences with findValue and findSale, and there is also the addSale function as well. If I open up page.js, I have my addSale, I have my findSale, and then I went ahead and commented out the charts again because we're not ready to use those yet. Now I need to come in here and right below the yearlyTotal, we're going to go ahead and create our new map. So I'm going to say const, and then we're going to say monthlySales. And then in that, I'm going to go ahead and say equals, and this is very similar to creating a set. But instead of a set, we're going to go ahead and create a map, so open close parentheses. That is now our monthlySales. Now in our new sale, as soon as we click on Add Sale, which happens as soon as the modal pops up, we enter in our information and we hit Add New Sale. In our addSale, we're going to go ahead and say monthlySales, and then we're going to .set. Remember, we're using set instead of add. Now I'm going to give this a title. I'm going to give it a key. How am I going to identify this later on? I'm going to go ahead and just do it as a string first of all. So I'm going to go ahead and say newSale. The newSale is going to go in and grab that value from our newAmount. We're still referencing that newAmount right there. It's going to grab that .value, and that's how we're going to go ahead and alert that value. But let's go ahead and just console.log this. I'm going to go ahead and go to monthlySales and then just hit Save. I'm going to go ahead and preview this now. Coming back into my project, I'm going to hit Refresh, go into Add Sale, go ahead and enter in October. Let's enter in the amount and then hit Add Sale. Well let's go into our Inspect Element here, and let's go into our console.log. Let's go into our console, and you can see we have the key being newSale and the value being 1200. So it automatically separates out for me, and that way I can go in and I can grab all the keys, I can grab all the values. Well it's a nice little handy function there. Let's go back in here. And instead of newSale here as a string, let's actually enter this and as a number. I'm going to hit Save here. Go back in here and hit Refresh and then let's add our new set, and then let's add our value here and click on Add Sale. Right-click here, and then you can see that it's has that 1 as the key and then 1200 as the sale. Let's say that I wanted to find a sale, and I wanted to actually go in and after I've entered in the sale, I wanted to find that specific



sale. So you can do that by actually using the get method. What I'm going to do here is `console.log`, and I'm going to say `monthlySales` and then `.get` and what you do inside of open close parentheses there is you basically type in the key. In this case, we're using the key as 1, and that will actually go in and get that value. Let's go back in here and hit Refresh, then let's hit Add Sale. Go into this and add in October for the month and then 1200 for the amount. Now if I right-click on this and going to Inspect Element and go over to here, it will add that. I'm going in and I'm actually logging that out when I add the sale. But let's go in and use that Find Sale function. Go back into the window here. And then you can see that it is logging out that 1200. It's going in and finding the key. And once it finds the key, it's logging the value there. That's a way that I can go in and I can get the value. Let's say I wanted to go in and delete that. So let's use this Fill Sales to actually delete. I'm going to go into my `index.html`, come up here to find where it says Fill Sales right there. And you can see I have this `fillValue`. I'm going to go ahead and leave that function the same here. Come back into `page.js` and let's create a function, open close parentheses, open close curly bracket there, and we're going to say `monthlySales.delete`, and we're going to go ahead and pass in the key. So that's how I go in and identify which one that I actually want to delete. I'll make sure that I actually `console.log` that as well just so I can see the value. We'll `console.log` `monthlySales`. And now let's hit Refresh. Okay, let's go ahead and enter in October 1200 for the amount. Click on Add Sale. Right-click and go into Inspect Window, go into our console, and there we go. It is inside of there. But now let's go ahead and hit Fill Sales, which will actually— Oh, let's right-click on that Inspect Window or Inspect Element, come in here, and you can see that it has no value. We've gone in and we added it, but we've also deleted it, and we're logging it just to make sure that it's been deleted. Okay, so that's how you can go in and you can delete it and just make sure that it has been deleted. Let's actually get rid of `findSale` right there, and let's just check to see if it has the number 1 as well. So we can also do a function that says, let's go ahead and `console.log`, and let's log to see if this monthly sales has the value of 1, the key of 1, `monthlySales.has` and then pass in the value of 1. That's the key that we're looking for. Let's go ahead and hit Refresh, and let's add in a value. So we'll go ahead and start with October, 1200, and then let's add the sale. Click on Find Sale, and we should get that `console.log` that it's been true. And yes, we have that here. That key does exist. So I can quickly go in and check to see if there is a certain key inside of there. Now I want to modify this a little bit differently because we're just setting a key there. But I actually want to modify it to set the key to be whatever the month is. All I have to do is actually come into `newMonth` right here. Come down to where I have this key right there. Go ahead and paste in `newMonth` and then just type in value. Whatever somebody enters in as the value of that month, that's what we're going to be checking, which means we're no longer checking for one here so we can go ahead and just get rid of those for now. It's basically going to set that, and it's going to set the key as the month and then the amount as the value. When I was working with sets, I was working with two different sets. But in this case, I can work with one set and be



able to store the key, the month as the key and then the value as the amount that the person entered it in. I'm going to go ahead and hit Save and then Preview. And let's enter in October here, and then let's hit 1200 and click on Add Sale and then right-click and go into Inspect Element there. And then let's go ahead and expand this out, and you can see the key is now set to October, and the value is now set to 1200. Now if I was to add in another one here and then go ahead and hit Add Sale, right-click and go into Inspect Element, go into here, now you can see that we have two here. Let's actually `console.log` the size. So I can actually get the property of the size. Then I can log that size. So I can say October 1200. I'll go ahead and hit Add Sale. And then let's say this one is now November, and this is going to be 1400. Hit Add Sale, right-click and go into Inspect Element, and then you can see that it now has alerted 1, and it has alerted 2 because that's the size. The first time we ran it, it only had a size of 1. The second time we ran it, it had a size of 2. That is a property that we can use to see how many different entries there are. But that's how you go in and how you add and how you remove and some of the other basic methods that you can do with maps. In the next video, we're going to be talking about iterating through a map and grabbing in the values and being able to work with that map data to be able to update our graphs.

Iterating through a Map

Now we're going to be talking about iterating through the map and how we actually go in and grab the keys, how we go in and grab the different values, and what we can do with those, and it's very similar to working with sets. There are the two different ways to be able to iterate. We have the `forEach` and we have the `forEachOf` as well. Now I'm going to come in here and under `addSale`, what we're going to do is we're actually going to update the labels first of all. So let's go ahead and uncomment out the bar chart here, and you can see that we have this `monthlySalesChart`. We have our labels, which is a blank array. We have our data, which is a blank array as well. Let's add our labels. So we're going to update our labels. Now I can easily do this by a similar method to what I was doing before. So I can say `monthlySalesChart.data.labels`, and then we're going to say `equals`, and we're going to use the `Array.from` just like we learned about in the first section here. And we're going to say `Array.from`, and then now we're actually going to just say `Array.from` not just `monthlySales`, but we want to grab just the keys. We can come in here and say `monthlySales.keys` open and close parentheses, and then there we go. Let's update that chart. So I'm going to come in here and say `monthlySalesChart.update`. We're going to call the method for `Chart.js` to be able to update the graph, and let's just do that for now. Let's go ahead and go in and preview our project. So I'm going to hit Refresh here, and I have that as soon as I update it. So I'm going to click on Add. I'm going to go in and say October and then say 1200 and then hit Add Sale. So you can see it automatically updates that. So let's go ahead and add one more here. So I'm going to say November, and



then we'll say 1400 here and then click on Add Sale. Well it automatically does that for me, so I don't necessarily need to go in and iterate through the keys just to grab the keys. I can just quickly do that by converting over the keys, but it's just a lot more simple than what I had to do before. All right, what we need to do now though is we actually need to go push in that data. We're going to use, for example, we're going to use a simple `forEach` just to log out to see if this is what you want it to do. So we're going to say `monthlySales`, and then we're going to go `.forEach` open and close parentheses. And then inside of those parentheses, we're going to go ahead and do a function, and then we're going to pass in the parameter of `sale`. That is going to be the data that we're going to work with, and then we're going to say `console.log`, and then we're going to log `sale`. Okay, so let's go ahead and preview this now. Come down here and click on Add Sale. We're going to go say October 1200 and then go ahead and click on Add Sale. Let's go into our Inspect Element, and you can see that it logs 1200. So I can get right into that sale. For each of the `monthlySales`, we're basically going to go in and grab that value. However, you might remember a problem that we had with the set is that data that we are working with is actually a string. When it comes in as the new amount, it is a string. So we actually need to go in and parse the integer of that so we actually get the number instead of the string. Now if I hit Refresh, visually it won't look much different here. If I enter in the amounts of 1200 and then go into Add Sale and then Inspect Element, it looks like we got a problem. We have our `parseInt`. Looks like I misspelled `parse`. Yep, I needed an `e` there so let's try this again. It's 1200. Click on Add Sale here. And now if I Inspect Element, there we go. This is blue. So it does let me know that it is a number and not a string by that text color. If I wanted to loop through and grab all those values, I can do that. But another way that I can do that is by using the `forEach`. So let's actually get rid of that, and I'm going to go in and just grab the value. So I'm going to say `for`, and then I'm going to inside of here say let amount of `monthlySales.values` where you just want to go in and grab the values. We're still parsing the integer, so that's great. And we just want to grab the values, make sure that that is plural there, and then open and close curly brackets there. And also, make sure that I actually close this with the parentheses there as well. I don't need a parenthesis at the end of that. It actually needs to be right here. It's going to go through each of those and then it's going to allow me to go in and update the amounts to the `yearlyTotal`. So we have our `yearlyTotal` up here. I'm going to say `yearlyTotal = amount + yearlyTotal`. So we're going to take the new amount and add it to whatever the current amount is, and then we're going to update that label, so we'll say `yearlyLabel.innerHTML =`, and then we're going to say `yearlyTotal`. That is going to be the new number. If I go ahead and hit Save and hit Refresh here and then click on Add Sale, you can see as soon as I add that sale, it's now updated that label. Let's go ahead and add another sale, so November and then 1100 and click Add Sale. It updates that automatically. It does the math for me. Now what we're going to do for each of that data amount, for that amount here, what I'm going to do is push that amount similar to what we did inside of our sets. If I just copy this code in here, what we're doing is we're



saying `monthlySalesChart.data.datasets.forEach` of them, so we're doing a `forEach` inside of here. We're going to take the dataset, and we're going to push in the amount right here. It's going to do that automatically for us, and then it's going to update the chart. Now if I come in here and hit Refresh and click on Add Sale, it's going to update the label, but it's also going to update the amount here. Now we'll have a problem, a similar problem to what we had before. If I come in here and say this is now 1500, notice that the first one actually gets set to 1200 and then a third one, which doesn't have a label gets set to 1500. I just can't see it. That's because I need to clear out the data. I need to clear out `yearlyTotal` as well. And then I need to clear out the data for this before I start adding new amounts. And I don't need to do it in the for loop there. I can actually come in here and say `yearlyTotal = 0` just to set that back to 0 there. And now I can actually come in here and copy the same code that we worked with inside of our sets, and this is just basically setting the Chart.js data back to a blank array. For each of those, we're going to set the data back to that blank array. Now if I come in here and hit Refresh and then click on Add Sale, go into this and start adding in our value here, it's going to add in that first value, but let's go ahead and add in the second value as well. We'll do 1500 here and then click on Add Sale, and you can see it automatically updates for us. In the next video, we're going to be talking about what the difference is because some questions might have popped up that's like why don't I just use objects for this type of thing? We're going to talk about what the difference between objects and maps are and why you'd want to use maps over objects..

Difference between Maps and Objects

So a question may have come up throughout this. If you're familiar with objects, you're familiar with arrays. Why are we not just using objects? Why would I actually want to go in and use maps? There are a couple big differences, and in this video, I wanted to explore what those differences are. The first one is being able to iterate so much faster through a map than you can through an object. As far as structure, you can go in and create a map, and you can have a key and a value for that map. For the object, you can go in and still do a key and a value for that object as well. But there is a difference in the key value. The key in the object, the keys must be simple data types. That means you can only have integers, you can only have strings, or you can only have symbols and really nothing more than that. For maps, you can have any data type. You can have primitives. You can have objects as well as a key. You can have the object. You can have arrays. You can have functions. You can pass in a spread operator for the array. Really, that is some of the differences is if you're wanting to work with more than those simple data types as a key. Another one is element order. So keep in mind that objects in arrays, you essentially can't guarantee the insertion order. It could be jumping all over the place. But with maps, you guarantee the insertion order. So if you wanted to go into first sale until the latest sale, you can



go through the map and iterate through that and make sure that you have that insertion order there. Another one is inheritance. A map is inherited from an object. That means that as a map, I can actually go in and get the methods and different properties of an object, but an object is not a child of a map. So you can't actually inherit some of those methods inside of an object, and so you don't get some of the methods that make it a little bit easier to work with like just being able to use `has` or being able to use `set` and `get` as far as setting the data of the map. So just things like that that you don't get that when you're working with a map, but you do get the methods from an object when you're working with a map as well. Now some of the other functional differences that I wanted to run through is, for example, creating. There are several different ways to be able to create an object. You can see that you can create a blank object. You can create a new object as well. But for maps, there is just one way to be able to create a map. So it simplifies the whole process and makes it more consistent. Getting, there's a couple different ways to be able to get the object data. But for a map, there's only really one way. You use the `get` method. For checking, you can do a bunch of different things to see if something exists and if it's equal to an object. But for checking inside of a map, it really comes down to that `has`. Does this have a certain object? And if it does, it'll result in `true`. If it doesn't, it'll result in `false`. And so it just kind of simplifies the checking a little bit more. For adding, you can add to an object in a couple different ways. But for adding in a map, it really simplifies it down to `set`. You basically `set` and you pass in the key and the value. For size, you have to basically say `Object.keys(obj).length` in order to get the object size. But for map, you just use a simple `map.size` in order to get the object size. And that's some of the main differences that you want to keep in mind when you're using maps over something like objects. You get access to these additional methods that you don't have with an object. It simplifies some of the code as well. It is faster to use. And so that's really some of the main differences why you'd want to use a map over an object. So in the next video, we're going to go ahead and dive into what some of the differences between a map are and a weak map, and then we'll go in to actually using the weak map and explore how to go in and create a weak map.

Understanding the Difference of Maps and WeakMaps

Now let's dive into some of the differences between a map and a weak map just so we're on the same page as far as when you would want to use a map and when you would want to use a weak map. And a lot of it is very similar to when you would want to use a set and when you would want to use a weak set. Keys must be objects in this case. You cannot have a key value as a primitive value, so you have to have it as an object. Now the objects similar to sets are actually held weakly. And so if it's there, it's there. If it's not there, it's not there. And they're not iterable, just like sets. You cannot iterate through a weak map. And they're also garbage collected. And so if it's not necessary for that to be stored inside of



data, it'll automatically free it up. So weak maps are also not enumerable. Enumerable means that they cannot be fully counted. So even though I created a map, if it's not being referenced, I can't count on that in that certain number. In maps, you can use keys and values and clear and entry and forEach and get and set and has. But for weak maps, you really can only use delete, get, and set and has, and that's really it. You're basically down to those different methods to be able to get, to set, to delete, and to check to see if the value is there. So let's actually go ahead and set up our weak map and work with some of the different methods that a weak map has. Now that we have a general idea of what a weak map is, let's just go in and explore the weak map here. I'm going to go ahead and create a variable here, and I'm actually going to place this inside of a block of code. And I'm going to go ahead and say let salesA, and we're going to create an object here because the key inside of a weak map needs to be an object. So let's just go ahead and create the object. And I'm going to say the object is going to have two different values, and that's going to be 1, 2. Now what I'm going to do is actually create a map, and we'll just start out with a map here just to explore the differences. But I'm going to say this is going to be a new map open close parentheses and then end that with the semicolon. And now I'm going to say map.set, and we're going to pass in the value of salesA. That is going to be our key. And now we're going to pass in the value of Hiking. Now inside of here because it's inside of curly brackets, I should not be able to actually console.log the map. However, because maps are not garbage collected in general, you can actually go in and you can console.log this. And so I can say console.log, and then I'm going to go ahead and console.log map. Now let's go ahead and preview this. I'm going to go ahead and hit Refresh, and then I'm going to right-click and going to Inspect Element. Now as soon as I do that, you'll notice that the map actually gets alerted here. I have my object and I have my Hiking. But again, I should not be able to do that because it's inside of here. One way to make sure that that does not happen and that it's garbage collected is I can come in here and change this to WeakMap. Instead of using Map, I can go ahead and say WeakMap, and then it will be garbage collected, and I should not be able to access it. I'm going to log this, first of all, inside of the brackets. I'm going to say console.log. And then inside of there, I'm going to say First and then add + and let's make sure we have a space, so we have a space in between the word, and we're going to say console.log. And then we're going to say Second, and then I'm going to say + and then salesA as well. You can see here the console.log, the first one should give me that information, but the second one will no longer actually work. So if I come in here and hit Refresh, you'll notice that it's not loading fully because I'm getting an error. If I go into Inspect Window and you can see that this error is essentially saying that I cannot find salesA because it's inside of these brackets. I don't have access to it. And if I go ahead and comment this out, let's go back in here and hit Refresh. And now if I inspect the elements again, now I will have access to that. So it is garbage collecting, so it's not available outside of that scope. So it's just a way to be able to work with your data more securely and make sure that it's not



available outside of a certain scope as well. So that's one way that you'd want to use a weak map.

Module Conclusion

Let's wrap up by talking about what we've learned about in the section. And first of all, we were introduced to maps. And so we talked about what maps are, how you can actually add and remove values from maps, how you can iterate through maps, and also the differences between maps and objects and the reason why you'd want to use maps and some of the additional methods that you get when using maps and then also the difference between maps and weak maps. What maps are and what weak maps are and some of the benefits that you get with WeakMap. That's really what we've covered. This section was designed just to cover maps and talk about some of the benefits to maps. In the next section, we're actually going to go back to arrays, but we're going to go back to a specific type of array, and that is called typed arrays. And we're going to talk about why you would want to use typed arrays, what typed arrays are used for, and actually start exploring typed arrays in the next section.

Exploring Typed Arrays

Module Introduction

So we talked about sets and we talked about maps and we talked about array functions in this course. We're going to wrap up this course by exploring typed arrays. Now typed arrays is a very specific type of array, and it is designed to handle direct memory allocation. So you can actually go in and store things inside of the memory and work with it that way, which helps being able to have your applications run faster. As web apps start to work more and more with large amounts of data, this might be useful as well. So we're going to go ahead and define what typed array is. We're not going to dive too deep into typed arrays in this section, but we're at least going to explore what it is. We're going to explore how you can start using it and how you can go in and start manipulating the typed arrays as well. So let's go ahead and first of all just talk about what a typed array is.

Defining Typed Arrays

Before we start working with typed arrays, let's go in and let's define what typed arrays are. So typed arrays are array-like objects that provide a mechanism for accessing raw binary data. And you can get straight to this data so you can actually work with this data quicker and easier. And if you have a web app or are



working with a web app that has to do with a lot of data, then typed arrays may be a good source for you. Let's talk about some of the benefits of using typed arrays. And the main benefit we've already talked about, but it bears repeating, is accessing raw binary data. You can get straight to that data. It then allows you to do faster performance and faster data manipulation as well and then strictly control data. So that means you can't put just any data inside of a typed array. It is controlled, and it depends on what type of view that you're actually working with. And there are a lot of APIs that support typed arrays that you can use in order to get this faster performance and access to this raw binary data. So some of the APIs that you may need to use or you may want to use are things like WebGL, Canvas, Web Audio API, XMLHttpRequests, Fetch API, Web Sockets, Web Workers, Media Source API, and File APIs. A lot of different APIs available that will support typed arrays, and you can start using them right away. So the key to the typed array structure is being able to actually create an array buffer. When you create this array buffer, you define how many bytes you actually want to work with inside of this array buffer, and you do that inside of your JavaScript file. The problem is though this JavaScript file cannot communicate directly to this array buffer. In order to communicate and get the functionality of an array, you actually have to create what's called a view. Now that view can contain different types of data. So in this case, we have a Uint8Array, or we have a Uint16Array, Uint32Array, Float64Array, and there's other types. Depending on the type of view that you create, you'll have different sets of structures in there that contain different types of bytes there. The key is how many bytes you actually initially create. So if I created 16 bytes, then that means with the 8Array, that means I have 1 byte each. And so that means I can have 16 different sections starting out with 0 going all the way up to 15, and I can store data in there. Now if it is going to be an Int8 array, that means I can only actually use certain types of numbers. Now if I'm going to do an Int16, that means I have 2 bytes each, and that means I can only have 8. And so if I'm using 2 bytes, I only have 8 to work with, and I can't go beyond, and I can't play something in like the 9 section or anything like that. If I'm working with an Int32 array, then I can work with higher numbers, but it's going to take up 4 bytes each, and that way I only have actually 4 spots to work with. If I'm going to do a Float64, that gives me a greater amount of numbers, but it lessens the amount of spaces that I have to work with. So in this case, with only 16 bytes, I can only split that up into two different spots there. And then inside of my JavaScript, I actually communicate to these views. So I've defined what type of number I'm working with, I've also defined the spot that that number is going to be stored inside of my array, and then I do all of that inside of my JavaScript. So hopefully that helps you actually understand how to create the arrays and how to split up the arrays, depending on the types of numbers that you're using. The key though is remembering that you're communicating to these different spots. And so if I wanted to place a number that's this 8Array here, then I would actually have to define what spot it's going to go in. It automatically creates those spots for me, and all of them start out as 0. But if I want to update a number in the second spot, then I would have to make sure that I update it in that spot, or I can



update it in the 5th spot, or, in this case, it's the 6th spot because it starts out at 0. What type of numbers can I place in each of these types of arrays? Well, that depends on the array that you're actually using. So these are the different types of arrays that I can create. So if I'm going to use an `Int8` array, then I have from -128 to 127. If I try to place anything like 128 or anything above there, it won't happen. I'll run into some different errors there. Now the size in bytes, if I stick within those numbers and within that array, is only 1 byte per section essentially. So then I have `UInt`, which gives me 0 to 255, `UInt8Clamped`, which means that if it goes above 255, I won't get that error, but it's just going to clamp it and keep it at 255 in case it does go beyond. That's maybe something that's kind of a safety mechanism that you want to keep in there. `Int16`, you have -32768 to 32767. So it just depends on what type of array that you actually want to use and the types of numbers that you're actually going to store in that array. And it goes all the way up to `BigUnassignedInteger64Array`. And you'll notice also the `Float64Array` if you're working with those numbers, but it does take up 8 bytes with each of these bigger numbers. So that's the concept. We're going to go ahead and start creating an array. We're going to create a buffer, and then we're going to go in and create some views in order to communicate to that. Now the view will then store that inside of that buffer. But I can't actually go into that buffer and manipulate anything about that buffer. I have to go into the view and manipulate the view, and then it will automatically manipulate and update the buffer from there. So that'll make a little bit more sense once we actually dive in and start working with these typed arrays.

Typed Arrays vs. Standard Arrays

So one more thing that I wanted to cover before we dive in and start working with these arrays is what are typed arrays versus standard arrays? Why would I use typed arrays, and why would I use standard arrays? And so let's compare the different two types of arrays. So in standard arrays, we accept pretty much any data type. We're able to use standard variable storage. We're also able to use standard processing as well and being able to iterate through it as well. With typed arrays, we're restricted on the data types. We only have certain numbers depending on what type of view array that you actually create. So if you do the `UnassignedInteger8Array`, you have certain types of numbers that you have to stick with within that array. One of the benefits though as you have access to this binary data is it becomes a lot faster to actually work with this type of array. And so if you're using a lot of data that's just numbers, well then a typed array may be a very beneficial thing for you to do because it's just so much faster as far as the processing being able to work with this. You have your APIs like the WebGL. You have the media APIs as well. So if you're doing a lot of that type of manipulation on really heavy coding there, then you may want to stick with that. Once it's in a view, it does act like an array. So you have all of the methods to an array and properties to an array, and so you can actually use it just like an array at that



point. And then you get access to those native APIs, so WebGL. You have Canvas as well. So if you're working with those different types of APIs, you have access to that, and you can take advantage of that and make it go quicker and perform quicker by using these typed arrays. So that's some of the differences between a standard array and a typed array. And in the next section, we're actually going to go in and build the buffer and show you how to build the buffer itself. And in the following section, we're going to go in and build the views and show you how to actually add data through the views.

Creating Typed Array Buffers

Let's first start out by creating this buffer. It's a pretty simple process to be able to create the buffer, but we need to be able to do that before we can start creating the views on top of that buffer and start adding the data. Keep in mind that we cannot manipulate the buffer itself. We have to create a view in order to do that. Download the sample project, and then we have this 01 Starter Project file. Make sure you download it for this section and make sure that you save your project from the last section so you don't mess that up as well. And so I'm going to come into this 01 Starter Project file, and I'm going to copy that and paste it onto my desktop. Now with that on my desktop, I'm going to go ahead and double-click on this and then double-click on the index.html. That's going to pull up my project here, and we're going to go in and open this project up inside of Sublime. So I have this project opened up inside of Sublime. I'm going to go ahead and open up the index.html. And notice if I come back into my window here, I don't have all those different sections. I've kind of simplified it a little bit more because we're just going to be working with this binary data. Now I'm going to come in here and actually go into my page.js, and I've gotten rid of some of the different functions. I've gotten rid of the yearlyTotal as well. We're mostly going to be working with the JavaScript inside of this section here. So now I'm going to go in and create my array buffer where it says Code goes here. Let's go ahead and get rid of that, and let's say let testBuffer, this is just a test that we're going to be working with, = new, and then we're going to say ArrayBuffer. So this is actually not an array, but this is an ArrayBuffer. Now we need to define the number of bytes that we're going to allow inside of this ArrayBuffer. I'm going to go ahead and say 16 because that's the examples that we've been working with, so we're going to go ahead and create that ArrayBuffer. That just defines inside of the memory that we have these 16 bytes that we can work with. So at this point, I can't really do much with it. We need to be able to create a view so I know what type of data is acceptable inside of this array. But we can check to see if the byte length is a certain number. So I could come in here and run an if statement here and say if testBuffer.length or byteLength, we're looking for byte length exactly, so byteLength, is equal to 16, then we're going to say alert Yes. That's really all that we're going to do. We're just going to test the byte length there. So if I come in here and hit Preview, you'll notice that it says alert Yes. So if you



needed to do a quick, if you have a lot of different `ArrayBuffers` in here, and you needed to do a quick check to see if this is the right byte length, then you can go ahead and do that as well. I'm going to get rid of that if statement there. And now in the next video, we're going to talk about actually creating a view on top of that `ArrayBuffer`.

Accessing Typed Arrays with Views

Now that we've created the buffer, we're going to go ahead and actually create the view so we can access the data and update the data. And there's different ways that we can actually do this. So I'm going into the code, and we're going to explore a couple different ways. So let's just go ahead and create our data here, and I'm going to say `let view1 =`, and we're going to say `new Int8Array`. This is how we define it as far as what type of integers are going to be placed inside of this buffer. We're going to say that this is going to be an int array, and we are going to go ahead and pass in `testBuffer`. We have to pass in what our `ArrayBuffer` is, and that's how we know how many bytes we have to work with. Now that I have that, I'm going to go ahead and type in `view1`, and I'm going to choose the slot. You've just created 16 different slots inside of this `ArrayBuffer`, and now I want to go in and update the first one. So within these brackets, I'm going to say `0`, and that will update the first one, and I can define it by saying `32`. Now keep in mind the different amounts that are available inside of my `Int8`. I believe it was `-128` to `127`. So I can't really go above that. So I'm going to go ahead and `console.log` this, and now I'm going to pass in `view1`. So let's go ahead and preview this. Let's hit Refresh. Nothing's really updating on the screen here. We're just kind of working with the data in the background. I'm going to right-click and go to Inspect Element. It logs all of the 16 different arrays. Because I created this `Int8Array`, it has logged all of them. And the first one, which is `0`, has now been logged as `32`. All the rest of them because they've been created are now `0`. They're set to `0` by default. That's something to keep in mind and something important to remember there. So I can come in here, let me end this with semicolon, but I can also come in here and say `view` brackets `view1` and then say `2` and then update that with a number as well. But keep in mind that if I actually try to update this beyond that number, so `Int8` only allows me to go up to `127`. However, if I try to go up to, let's say, `132` here, watch what happens. We're going to alert `view1` again. I'm going to hit Save and hit Refresh and then go into Inspect Element. Now I'm going to go into here and then expand this out, and you'll notice it automatically jumps me back to `-124`. That is because we're trying to pass in a value that that is not accepting. And so that's why you have to work with the correct data, making sure that you don't go beyond that. If you wanted to make sure that it goes beyond that and does some error checking, then you may want to use the `ClampedArray` instead of using `Int8array`. So just a couple things to keep in mind there. Okay, so now let's actually change this. Instead of `8`, we're going to change this to `16`. And if I go ahead and save that and preview



it, let's go ahead and right-click and go into Inspect Element. And you'll notice as soon as I alert this, I don't have as many to work with. Now the 16 bytes are actually being taken up by 2 because we're using this integer 16. Now we can also place a DataView on top of this array as well that we can actually work with separate. So if I come in here, I could say let view2 =, and then I can do a new DataView, and I can pass in that testBuffer as well. So instead of defining the exact amount right here, I can create just a generic DataView there. Now as soon as I have created that generic DataView, I can now set that data with certain types of values. So I can say view2.set, and I'm using a method called setInt8, and this will set the view to 8 here, and it will allow me to pass in the value. I need to pass in two parameters. The first one is the position, and the second one is the amount that we want to actually update it with. So let's go ahead and do that. And so let's just go ahead and console.log, and we're going to go ahead and console.log view2. It's kind of a different way of doing it. Instead of defining the integer there, we're defining the integer as soon as we set it. So let's go ahead and hit Refresh and then go to Inspect Element. And you can see that the first one is that Int16, and there's all of our different values here. The second one is just our data view, but if I expand that out, I can actually get to that data, and you can see that the byte length is set to 16 as well. But notice what's actually happening. When I set the second position to 43, it's actually updated that inside of the array inside of the buffer because I'm working with the data. I'm not just working with the view. But the view is going through and actually passing in that data inside of here. So just keep that in mind that I'm creating a new view, but I'm still working with that same ArrayBuffer data, so it's still updating that as well. A couple other things that I want to do here. Let's go ahead and actually just take that out here, and we are are going to just create a variable and say num = view2.getInt. We're going to go ahead and get the integer, and we're going to go ahead and get the position 2. Make sure we do getInt8 because that's how we set it was the Int8 there, and I'm going to go ahead and log that as well. So we'll say console.log, and then we're going to go ahead and pass in num there. Now if I hit Save and then Preview, let's go ahead and inspect that. And you can see that we're actually getting an error, so let's double-check this. It looks like I spelled console wrong. So let's try this one more time, hit Preview, go ahead and go into Inspect Element, and then let's preview that, and you can see that we're getting that number of 43. So I'm going into that position, and I am updating the value of 43, and then I'm getting that position, and I'm alerting that position here or logging that position there. All right, so let's try something else here. What I want to do is actually create another view here. So let's say let. So let's do this one as view3, and then we're going to say new DataView. Make sure that we have a space there DataView, and then we're going to pass in that testBuffer. Let's go ahead and create another variable and store this data inside of num2. This is going to be our number 2. That is going to be equal to the view3, get the integer 8, and we're going to say this is going to be 2 as well. Let's do console.log, and we're going to log num2. So even though I'm creating a new view, I'm still going to be able to access that data. So if I go ahead and right-click here and go into



Inspect Element, oh, I forgot an equal sign there, so let's do equals. Make sure we have a semicolon there and come back in here and hit Refresh. Right-click and go into Inspect Element, and you can see here that we're still getting that. So even though I'm actually creating a new view, I'm still working with that same type of base data. So we're going to do something a little bit different. We're actually going to go in, and we're going to define how many numbers we actually want to work with inside of a certain view. So we have these 16 bytes and so these 16 different positions here, and I'm going to go in here, and I'm going to create a view. So let's go ahead and say `view4`, and we're going to say equals, and then we're going to go ahead and say `new DataView`, and we're going to pass in the `DataView` and say this is going to work with the `testBuffer`. But we're going to actually go in and we're going to start at position 7. So just like we've done here, we have this `testBuffer`. But by adding a couple more parameters, we can say that we're going to start at position 7, and we're going to move over 3 positions. So inside of this view, I really only have three positions to work with now. So I'm only updating it from position 7 to the next 3 of those positions. And so now I can come in here and I can say `view4`, and then I can say `setInt8`, `setInt8`. And then we're going to go ahead and say we're going to set the position of 0 because we're only working with those three positions. So that first position, which is position 7, is now, in this case, the position of 0. So we're going to say 54, and then we're going to say `view4.setInt8`. We're going to update position 1. That is now going to be 23. And we're going to go ahead and `view4.setInt8`. And then we're going to update position 2, and that is now going to be 32. So now from here, let's go ahead and get those values and we're going to alert those values. So I'm going to say `let numAlert =` and then `view4.getInt8`, and then we're going to go ahead and get the integer 0, and then we're going to `console.log`. Oh, I forgot the `e` there. Then we're going to say `numAlert`. So let's go ahead and preview that. Right-click, go into Inspect Element, and you can see I get that 54. So even though I'm getting that 0 and I'm updating the 0, it's actually at that 7th position. So I'm still working with that same `ArrayBuffer`, and I'm still working with that same code. So that's how you start working with typed arrays. And when you start getting into other APIs, you may be working with these different types of arrays in different ways, and so I wanted to explore some of that inside of this video. In the next video, we're going to go ahead and wrap up everything that we've learned, talk about what we've learned in this section, as well as talk about what we've learned in this entire course.

Module Recap

So let's wrap up this section by talking about what we've learned. We dove into an introduction to typed arrays, what it is, how it works, and how you can actually take advantage of it. We also talked about typed arrays versus standard arrays and the differences between those different arrays. We also talked about creating array buffers and what array buffers are, as well as being able to access



typed arrays with views, how you can't actually access the data with buffers or anything like that. So that's really what you need to know to get started with types arrays, and there's a lot more that you can dive in with typed arrays. This was designed mostly just to be an introduction to what typed arrays are and how to initially start working with typed arrays. That is what we have covered in this module. In the next video, let's quickly review what we've learned throughout the course and talk about some next steps.

Course Conclusion

Let's quickly recap what we have learned in this course. We started out by talking about some more advanced methods that you can use when you're working with arrays. We then covered new collection types like sets and maps and how to work with each of those data collections. We then finally finished off with typed arrays and how to use typed arrays. You may be wondering where do I go from here? Everything that we have learned about is great for working with data, but you may have noticed that every time I hit Refresh in the browser, the data went away. A great next step would be to learn how to work with data that stays even after you hit Refresh, like local storage or pulling data from a database. Sets, maps, and typed arrays are still useful when working with databases and local storage. So learning how to store the data in a database is a good next step for your progression. But I wanted to thank you for taking this course and making it all the way to the end. If you have any feedback on how I can improve this course, please don't hesitate to share. Thanks, everyone.